

Docker

WPC

2025 年 9 月 13 日

目录

1	Docker Engine	2
1.1	介绍	2
1.2	镜像	2
1.3	容器	3
1.4	网络	3
1.5	存储	4
2	容器部署	5
2.1	镜像构建	5
2.2	Docker Compose	5

1 Docker Engine

1.1 介绍

Docker Engine 由 dockerd (Docker Daemon)、containerd、containerd-shim 和 runc 这几个关键模块组成。

- dockerd: 负责接收 CLI 或 API 请求, 功能包括管理镜像、身份验证和网络等, 客户端与 Docker daemon 之间通过本地 Unix Socket 通信, 路径为: `/var/run/docker.sock`。
- containerd: 以 daemon 的方式运行, 负责容器的生命周期管理, 但它并不创建容器, 只是将 Docker 镜像转换为 OCI bundle。
- runc: runc 是 OCI 容器运行时规范的参考实现, 只负责创建容器。
- shim: 每次创建容器时都会 fork 出一个新的 runc 实例, 不过, 一旦容器创建完毕, 对应的 runc 进程就会退出, 此时, 相关联的 shim 进程就会成为容器的父进程, 保证容器在 containerd/dockerd 崩掉时仍可继续运行, 每个容器有独立的 shim 进程。

如果使用 systemd, dockerd 的日志会被写入 journald, `journalctl -u docker.service` 命令可以查看日志。如果需要调整守护进程的日志级别, 可以编辑配置文件 `daemon.json`, 启用调试模式并设置日志级别, 例如:

```
{
  "debug": true,
  "log-level": "debug"
}
```

其中 log-level 可选的值包括: debug、info、warn、error、fatal。至于容器级别的日志, 可以通过命令查看: `docker container logs <container>`。

image	container	network	volume			
ls	rm	inspect	ps	prune	create	

表 1.1: docker 常用命令

1.2 镜像

在 Docker 中, 镜像是由若干松耦合的只读层 (image layers) 组成的。我们可以通过以下方式了解镜像信息:

- 使用 `docker image inspect <image>` 可以查看镜像的分层结构。
- 使用 `docker search --filter "is-official=true" <repo>` 查找官方仓库中的镜像。

- 使用 `docker image prune -a` 可以移除所有没有被使用的镜像。

镜像在推送 (push) 和拉取 (pull) 的过程中, 为了节省网络带宽, 会对镜像层进行压缩。由于压缩会改变二进制内容, 因此, 为了校验, Docker 为每个镜像层同时维护一个分发列值 (Distribution Hash), 这是压缩后镜像文件的哈希。

本地镜像数据通常存储在: `/var/lib/docker/<storage-driver>`, 此外, 镜像有两种常见的标识方式: IMAGE ID 和 Digest, 前者是本地镜像的唯一标识符, 只在本地主机中有意义, 后者是镜像内容的整体哈希。

1.3 容器

在 Docker 中启动容器可以有多种方式, 具体取决于你希望容器以何种方式运行:

- 默认启动镜像中配置的应用: `docker container run <image> <app>`。
- 进入交互式 shell: `docker container run -it <image> /bin/bash`。
- 后台运行容器: `docker container run -d --name <name> -p <host>:<container>`。

容器的生命周期与其主进程 (PID=1) 绑定。当容器内的主进程退出时, 容器也会随之终止。在与容器交互时, 按下 Ctrl+P Q 组合键可以将终端 “分离”, 但容器本身不会停止。你可以通过以下命令再次进入容器, 开启一个新的 shell: `docker container exec -it <name> bash`。

为了在服务异常时保持容器的高可用性, 建议在运行容器时配置合适的重启策略:

- `always`: 无论因为什么原因停止, Docker 都会尝试重启容器。唯一的例外是容器被显式停止 (`docker container stop`)。注意: 当 Docker daemon 重启时, 所有处于停止状态的容器也会被尝试重启。
- `unless-stopped`: 与 `always` 类似, 但如果容器被手动停止 (`docker stop`), 即使 daemon 重启, 容器也不会再自动拉起。
- `on-failure[:max-retries]`: 当容器因错误退出 (退出码!= 0) 时, Docker 才会尝试重启。

1.4 网络

容器网络模型 (Container Network Model, CNM) 定义了 Docker 网络架构的基础组成要素。Libnetwork 作为具体实现, 负责提供控制层和管理层功能, 而网络驱动 (Driver) 则承担数据层的实现。Docker 已经封装了若干内置驱动, 在 Linux 包括 Bridge、Overlay 和 Macvlan 等。

在每台 Docker 主机上, 都会自动创建一个默认的单机桥接网络。在 Linux 系统中, 该网络名称为 bridge, 对应宿主机上的虚拟网桥 docker0。 `docker network inspect <name>` 命令可以查看网络的详细信息。

值得注意的是, Docker 为容器提供 DNS 解析功能, 不过默认的 bridge 网络并不支持通过 Docker 内置的 DNS 服务进行容器名称解析。DNS 服务器会记录容器在启动时通过 `-name` 或 `-net-alias` 参数指定的名称与容器 IP 地址之间的映射关系, 从而实现基于名称的访问。

Docker 内置的 Macvlan 驱动可以让容器直接接入宿主机所在的物理网络，使其表现得像一台独立的主机。要使用该驱动，宿主机网卡需要启用混杂模式 (promiscuous mode)。例如，假设需要将容器加入到 VLAN 100 中，可以创建一个基于 Macvlan 的网络：

```
docker network create -d macvlan \  
--subnet=10.0.0.0/24 --ip-range=10.0.0.0/25 \  
--gateway=10.0.0.1 -o parent=eth0.100 macvlan100
```

1.5 存储

非持久化存储的目录在 `/var/lib/docker/<storage-driver>/` 下，是容器的一部分，一般使用 Overlay2 驱动。要持久化数据要使用卷，默认情况下，Docker 创建新卷时采用内置的 local 驱动，创建的新卷在 `/var/lib/docker/volumes` 目录下。下面的命令创建一个新的独立容器，并挂载一个名为 bizvol 的卷：

```
docker container run -dit --name voltainer \  
--mount source=bizvol,target=/vol alpine
```

如果指定了已经存在的卷，Docker 会使用该卷，否则 Docker 会新建一个卷，要注意的是即使容器被删除，卷依旧存在。

2 容器部署

2.1 镜像构建

Dockerfile 文件指导镜像构建，内容类似这样：

```
FROM node
LABEL maintainer "test@mail.com"
ADD . /src
WORKDIR /src
RUN npm install --production && npm run build
EXPOSE 8080
ENTRYPOINT ["npm","start"]
```

之后可以构建镜像和推送到 hub:

```
docker image build -t <image-name> .
docker image tag <current-tag> <docker-id>/<current-tag>
docker image push <docker-id>/<current-tag>
```

为了精简镜像大小，可以使用多阶段构建，例如：

```
FROM node:latest AS storefront
...
FROM maven:latest AS appserver
...
FROM java:8-jdk-alpine AS production
...
COPY --from=storefront /usr/src/atsea/app/react-app/build/ .
WORKDIR /app
COPY --from=appserver /usr/src/atsea/target/AtSea-0.0.1-SNAPSHOT.jar .
...
```

重点是COPY --from指令，仅复制生产环境相关的应用代码。

2.2 Docker Compose

多数的现代应用通过多个更小的服务互相协同来组成一个完整可用的应用，比如一个应用可能由 4 个服务组成：Web 前端、订单管理、品类管理、后台数据库。部署和管理繁多的服务是困难的，而这正是 Docker Compose 要解决的问题。

Docker Compose 使用 YAML 文件来定义多服务的应用。YAML 是 JSON 的一个子集，因此也可以使用 JSON。示例文件：

```
version: "3.8"
services:
  web-fe:
    build: .
    command: python app.py
    ports:
      - target: 5000
        published: 5000
    networks:
      - counter-net
    volumes:
      - type: volume
        source: counter-vol
        target: /code
  redis:
    image: "redis:alpine"
    networks:
      counter-net:

networks:
  counter-net:

volumes:
  counter-vol:
```

启动应用命令是`docker-compose up`，使用`-d` 参数在后台启动应用，默认使用文件名 `docker-compose.yml`，也能用`-f` 参数指定其他文件。